DA5402 MLOps Project Report

Topic: Financial News Analysis

Name: Pranav Rao Roll No: DA25M022

Problem Statement

This application performs an analysis of Financial News in two methods:

1. Market Topic Classification - Identifying which part of the financial sector the news pertains (example - Central Banks / Politics / Energy / Company)
2. Market Impact Classification (Sentiment Analysis) - Identifying the broad impact the topic of the news will have on the market (Bearish / Bullish / Neutral)

The application accepts a news segment as a file (txt or image) or as text input.
It uses distillBERT finetuned on the corresponding tasks.

Task 1: Sentiment

Dataset

Training Dataset

The dataset was chosen from takala/financial_phrasebank at HuggingFace.
Some statistics of the dataset are:

Label	No
neutral	0.614
positive	0.252
negative	0.134

4840 sentences
5 - 8 annotations per sentence
Train Test Valid Split : 1 : 0.25 : 0.25

Evaluation Dataset

Newspapers:

1. TOI
2. Wall Street Journal
3. Washington Post
4. New York Times
5. Pioneer
6. Telegraph
7. Economic Times
8. Statesman
9. Tribune
10. Business Standard

Models

Three models were trained for the task:

1. huawei-noah/TinyBERT_General_4L_312D
2. distilbert/distilbert-base-uncased
3. CardiffNLP/twitter-roberta-base-sentiment

Hyperparameters:

1. Learning Rate : 2e-5 to 4e-3
2. Epochs : 5 - 15
3. Weight Decay : 0.01

Metrics

ML Metrics

1. F1 (Macro, Micro, Weighted)
2. Precision (Macro, Micro, Weighted)
3. Recall (Macro, Micro, Weighted)
4. Matthews Correlation Coefficient
5. Accuracy

Training Details

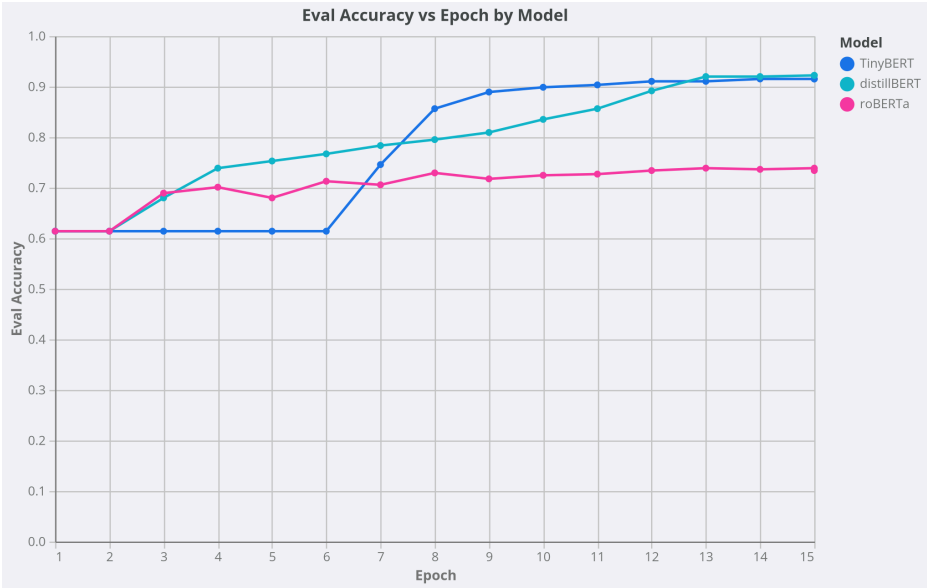
Optimizer used : AdamW
GPU - NVIDIA GeForce GTX 1650 Ti with Max-Q Design

Outputs

Overall:

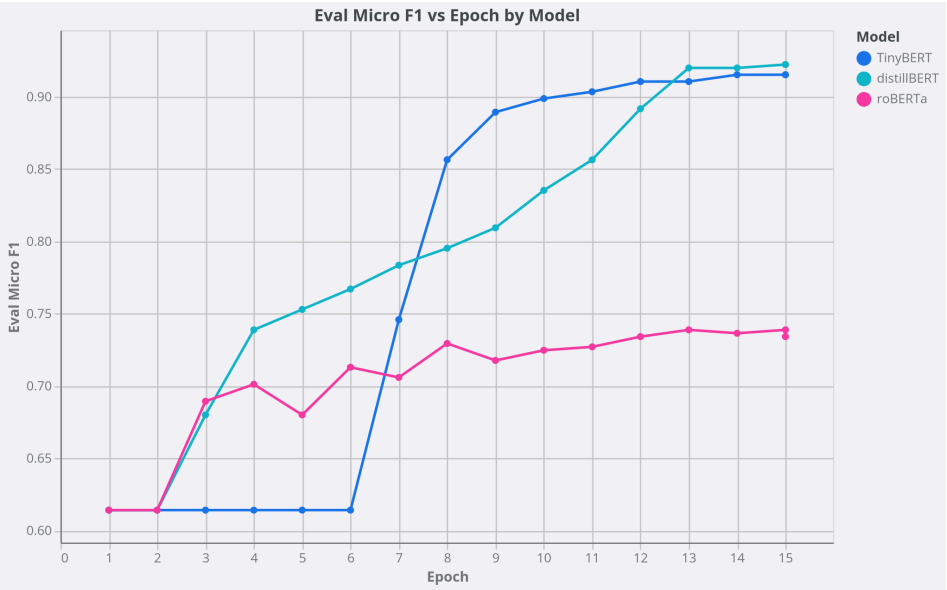
Compare by	Sentiment_Train_hua...	Sentiment_Train_rob...	Sentiment_Train_dist...
Model Metrics			
eval_accuracy	0.92	0.73 -0.18 ↓	0.92 +0.01 ↑
eval_brier	0.29	0.37 +0.08 ↑	0.16 -0.14 ↓
eval_ece	0.31	0.08 -0.23 ↓	0.12 -0.19 ↓
eval_loss	0.57	0.66 +0.09 ↑	0.30 -0.27 ↓
eval_macro_f1	0.90	0.59 -0.30 ↓	0.90 +0.00 ↑
eval_macro_precisi	0.90	0.61 -0.29 ↓	0.91 +0.01 ↑
eval_macro_recall	0.90	0.58 -0.31 ↓	0.89 -0.00 ↓
eval_mcc	0.84	0.49 -0.36 ↓	0.86 +0.01 ↑
eval_micro_f1	0.92	0.73 -0.18 ↓	0.92 +0.01 ↑
eval_micro_precisio	0.92	0.73 -0.18 ↓	0.92 +0.01 ↑
eval_micro_recall	0.92	0.73 -0.18 ↓	0.92 +0.01 ↑
eval_pr_auc_macro	0.94	0.63 -0.32 ↓	0.95 +0.01 ↑
eval_pr_auc_micro	0.95	0.82 -0.13 ↓	0.97 +0.02 ↑
eval_pr_auc_weight	0.96	0.76 -0.19 ↓	0.97 +0.01 ↑
eval_runtime	1.05	12.64 +11.59 ↑	6.43 +5.38 ↑
eval_samples_per_	405.41	33.63 -371.78 ↓	66.08 -339.34 ↓
eval_steps_per_sec	25.76	2.14 -23.62 ↓	4.20 -21.56 ↓
eval_weighted_f1	0.91	0.72 -0.19 ↓	0.92 +0.01 ↑
grad_norm	1.98	9.53 +7.55 ↑	1.39 -0.59 ↓
learning_rate	0.00	0.00 -0.00 ↓	0.00 -0.00 ↓
loss	0.88	0.70 -0.18 ↓	0.66 -0.22 ↓
total_flos	35361467596530.00	648821757... +6134602... ↑	326662410... +2913009... ↑
train_loss	0.79	0.62 -0.17 ↓	0.54 -0.24 ↓
train_runtime	177.07	2565.51 +2388.44 ↑	1285.49 +1108.42 ↑
train_samples_per_	107.84	7.44 -100.39 ↓	14.85 -92.98 ↓
train_steps_per_se	0.42	0.03 -0.39 ↓	0.06 -0.37 ↓

Accuracy:

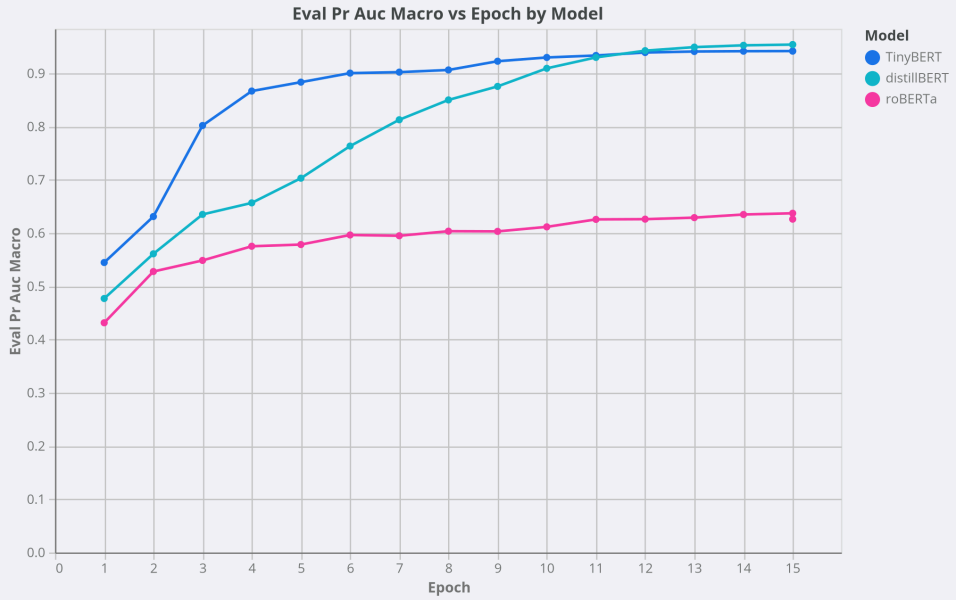


Macro F1:

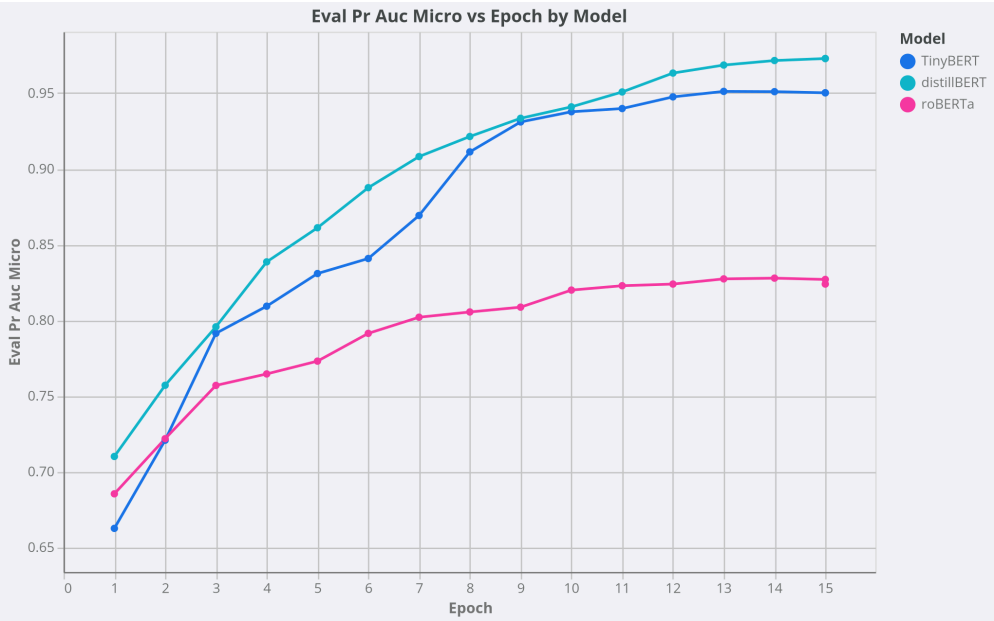
Micro F1:



Macro PR AUC:



Micro PR AUC:



Task 2: Topic Modelling

Dataset

Training Dataset

The dataset was chosen from zeroshot/twitter-financial-news-topic at HuggingFace. Some statistics of the dataset are:

Labels	No
Analyst Update	0.016
Company | Product News	0.208
Currencies	0.009
Dividend	0.022
Earnings	0.058
Energy | Oil	0.032
Fed | Central Banks	0.05
Financials	0.037
General News | Opinion	0.09
Gold | Metals | Materials	0.004
IPO	0.003

Legal | Regulation	0.029
M&A | Investments	0.028
Macro	0.106
Markets	0.03
Personnel Change	0.029
Politics	0.058
Stock Commentary	0.125
Stock Movement	0.048
Treasuries | Corporate Debt	0.019

Evaluation Dataset

Newspapers:

11. TOI
12. Wall Street Journal
13. Washington Post
14. New York Times
15. Pioneer
16. Telegraph
17. Economic Times
18. Statesman
19. Tribune
20. Business Standard

Models

Three models were trained for the task:

1. huawei-noah/TinyBERT_General_4L_312D
2. distilbert/distilbert-base-uncased

Hyperparameters:

1. Learning Rate : 2e-5 to 4e-3
2. Epochs : 5 - 15
3. Weight Decay : 0.01

Metrics

1. F1 (Macro, Micro, Weighted)
2. Precision (Macro, Micro, Weighted)
3. Recall (Macro, Micro, Weighted)
4. Matthews Correlation Coefficient

5. Accuracy

Training Details

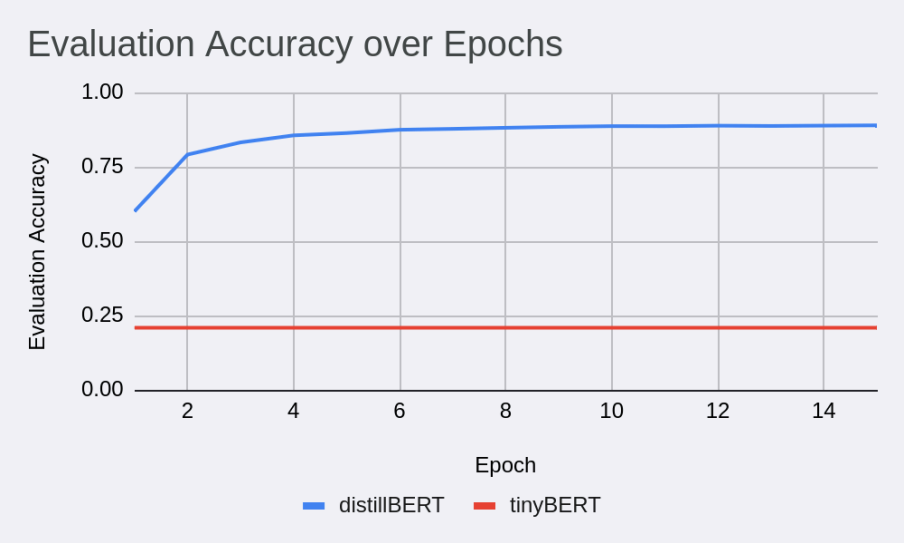
Optimizer used : AdamW
GPU - NVIDIA GeForce GTX 1650 Ti with Max-Q Design

Outputs

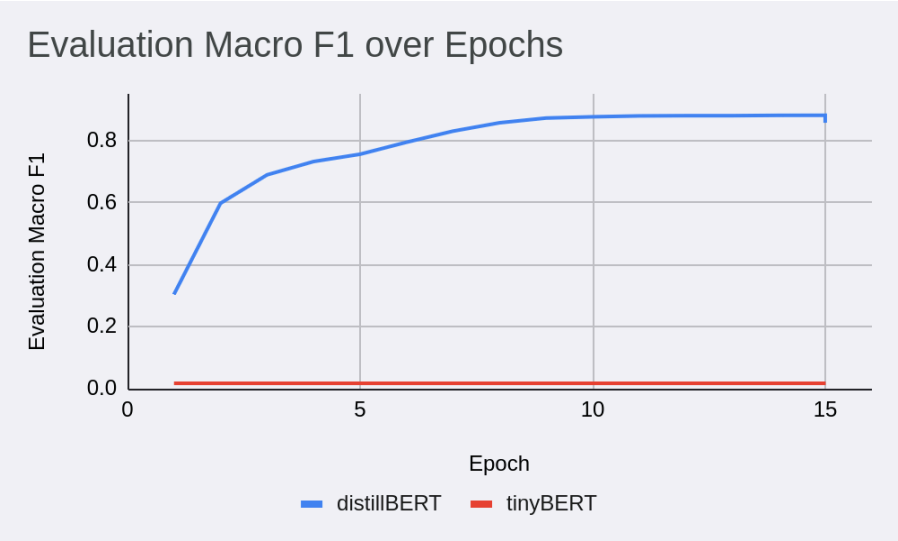
Overall:

Compare by	Topic_Train_distilbert/...	Topic_Train_huawei-n...
Model Metrics		
eval_accuracy	0.88	0.21 -0.67 ↓
eval_brier	0.19	0.90 +0.72 ↑
eval_ece	0.04	0.00 -0.04 ↓
eval_loss	0.46	2.61 +2.15 ↑
eval_macro_f1	0.86	0.02 -0.84 ↓
eval_macro_precisi	0.87	0.01 -0.86 ↓
eval_macro_recall	0.85	0.05 -0.80 ↓
eval_mcc	0.87	0.00 -0.87 ↓
eval_micro_f1	0.88	0.21 -0.67 ↓
eval_micro_precisio	0.88	0.21 -0.67 ↓
eval_micro_recall	0.88	0.21 -0.67 ↓
eval_pr_auc_macro	0.92	0.05 -0.87 ↓
eval_pr_auc_micro	0.93	0.14 -0.79 ↓
eval_pr_auc_weight	0.94	0.10 -0.84 ↓
eval_runtime	76.98	12.92 -64.05 ↓
eval_samples_per_	55.19	328.71 +273.52 ↑
eval_steps_per_sec	3.46	20.58 +17.13 ↑
eval_weighted_f1	0.88	0.07 -0.81 ↓
grad_norm	0.41	0.23 -0.17 ↓
learning_rate	0.00	0.00 +0.00 ↑
loss	0.04	2.61 +2.57 ↑
total_flos	4064019242099520.00	440301334... -36237179... ↓
train_loss	0.40	2.62 +2.21 ↑
train_runtime	12050.65	1701.28 -10349.37 ↓
train_samples_per_	15.86	112.34 +96.48 ↑
train_steps_per_se	0.06	0.44 +0.38 ↑

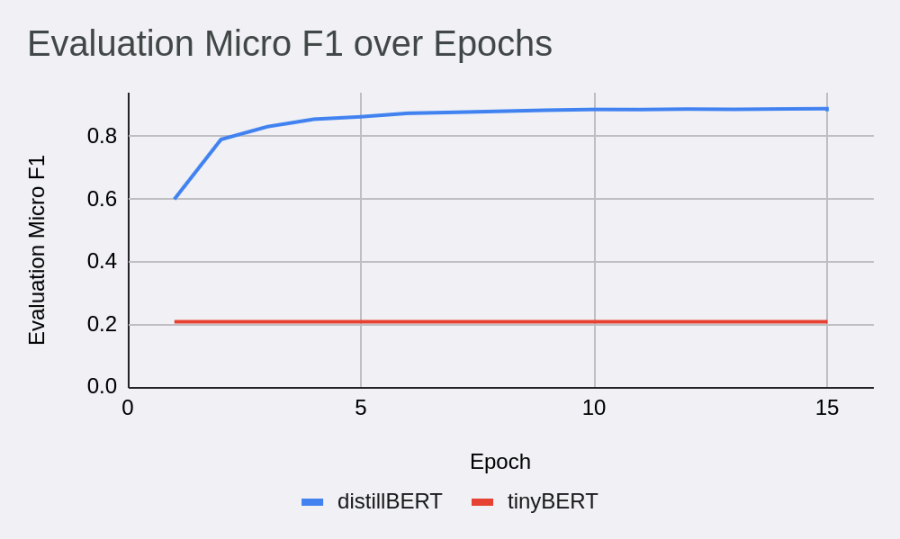
Accuracy:



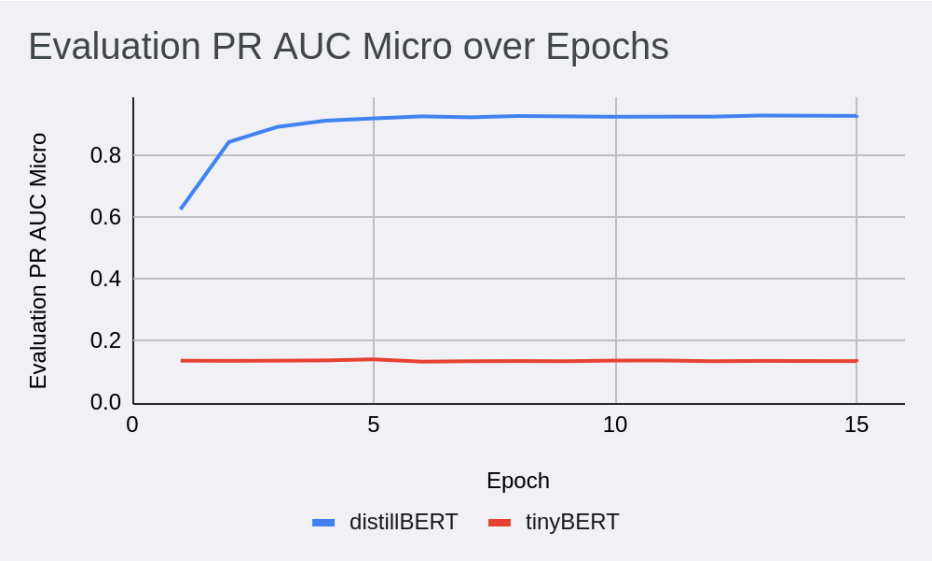
Macro F1:



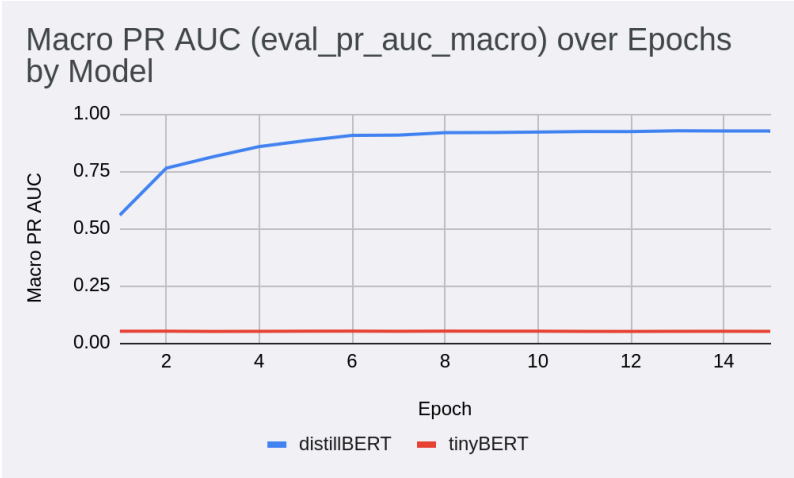
Micro F1:



Micro PR AUC:

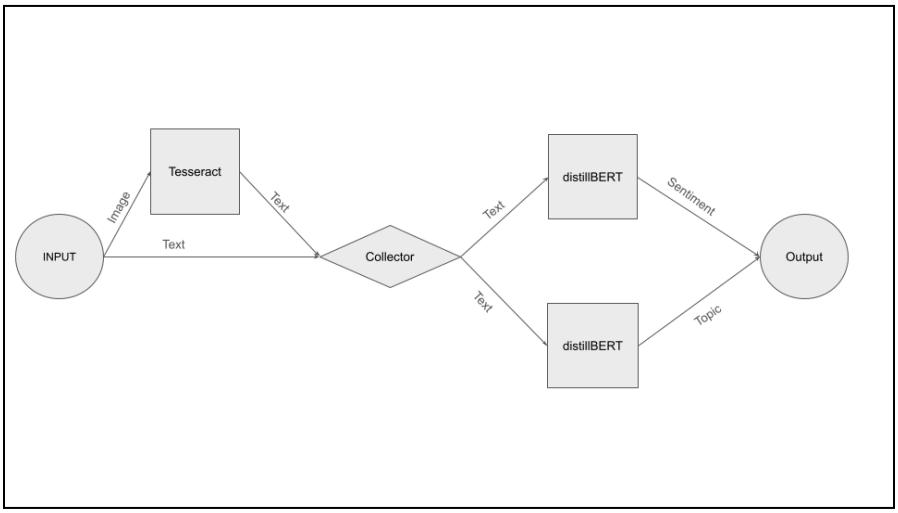


Macro PR AUC:

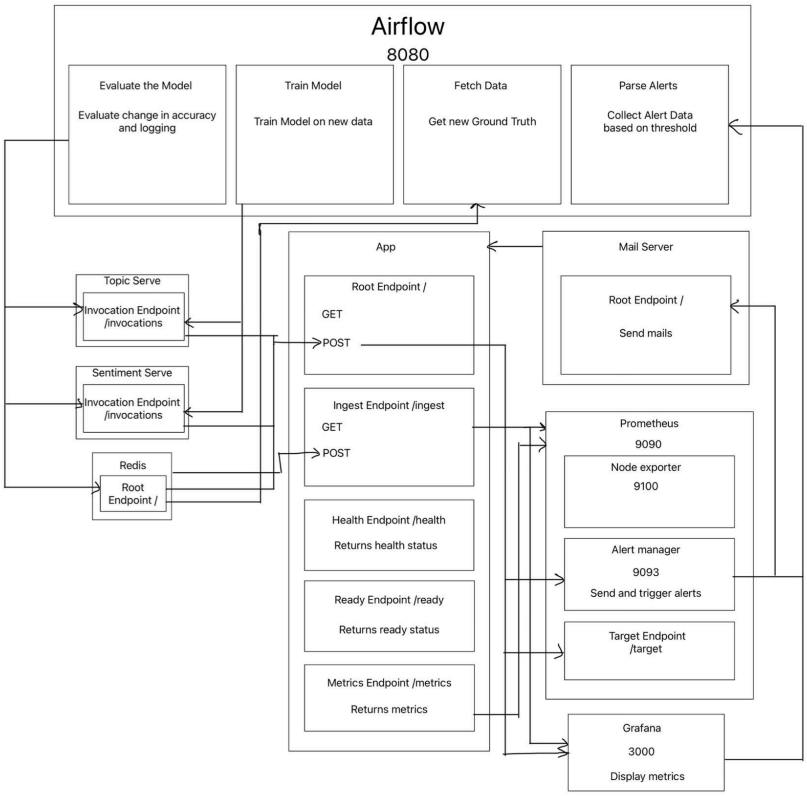


Application

Basic Inference Flow diagram:



High Level Design Diagram:



The software follows the functional paradigm, as there was less need for reusable components. It follows the standard Pythonic coding style with documentation, logging and exception handling.

The Flask application acts as a middleman between the frontend and the MLFlow models - cleaning the inputs and displaying the outputs. There are two threads being run - one for each task

Version Control is being done by DVC and Git.

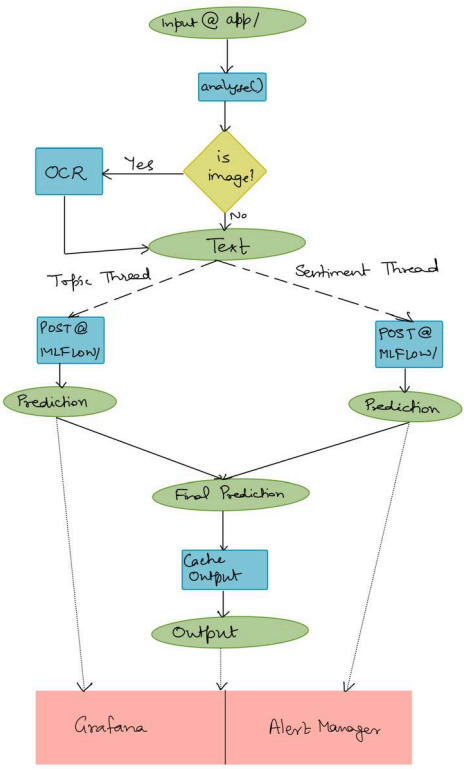
Yes - Does the low-level design (LLD) document clearly specify the API endpoint definitions with the respective I/O specifications?

No - Do you have a separate UI screen to visualize the machine learning pipeline, including the data ingestion/engineering pipeline?

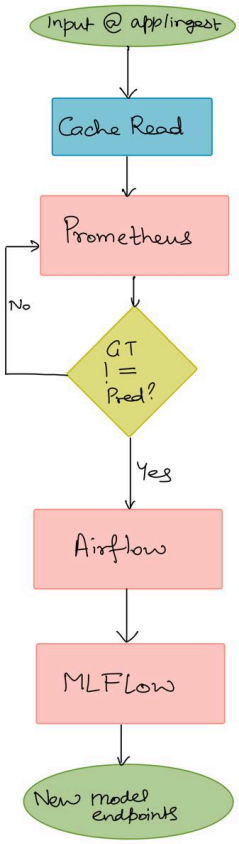
The Airflow DAG is for retraining.

Recorded by Grafana - What is the speed and throughput of your pipeline?

Flow of the Inference pipeline:



Flow of the ingest pipeline:



UI

Metrics

1. Application Health
2. Total no of inference requests
3. Total no of exceptions
4. Total no of active requests
5. CPU Utilization & Context Switch
6. System and App memory Utilization
7. Disk Read & Write
8. Data Drift in both Tasks
9. Current Inference input data distribution
10. Change in distribution due to drift
11. Inference Latency (sec)
12. Task based input class recording
13. Model Reliability
14. Ground truths
15. Prediction Correctness
16. Model Accuracy Decay

Tests

Yes - Is there a Test plan?

Yes - Is there a Test report attached that quotes the number of test cases, passed, and failed?

Yes - Did the software meet the acceptance criteria upon testing?

Alerts

1. High Host CPU Load
2. High Inference Latency
3. Application Down
4. High App Error Rate
5. Specific Model Down
6. Severe Data Drift Topic
7. Severe Data Drift Sentiment

Model Training and Experiment Tracking

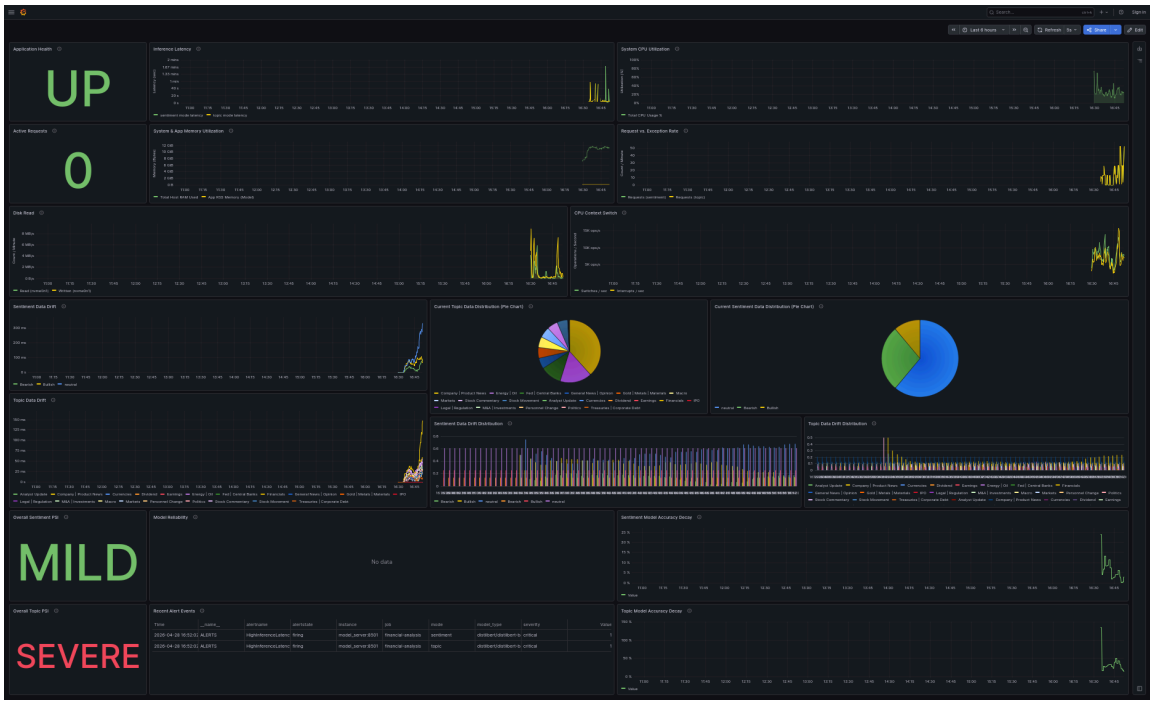
MLFlow does the experiment tracking and model serving. It also provides MLProject for reproducibility. We log the metrics as described in the "ML Metrics" section. The MLFlow dashboard shows various runs and experiments. Artifacts tracked are models, confusion matrices, system and model metrics.

Tech Stack

Version Control - Git & DVC
Application Building - Flask
Containerise - Docker
Experiment Tracking - MLFlow
Logging
Model & Datasets - HuggingFace
(Post /) Production Metrics - Prometheus
Alerts - Prometheus
(Post /) Production Metrics Dashboard - Grafana
Model Metrics - MLFlow
Model Serving - MLFlow
Application Orchaestration - Airflow
Ground Truth Logging - Redis

Post Production

Prometheus-based monitoring. It records the Business Metrics. It monitors all the App's metrics. Grafana Dashboard shows the metrics in NRT. It triggers alerts based on the thresholds. Based on the data drift thresholds, the Airflow DAG triggers the fetch of new recorded ground truth data, trains a new model instance, and redeployed.



Data Ingestion

Data Ingestion pipeline is the Airflow DAG for retraining the model for data drift. CI is done by dvc repro.

dvc repro - How is the continuous integration implemented using DVC?

Yes - Is there a DVC DAG representing your CI pipeline?

Ports

1. Topic Model Serve - 5003
2. Sent Model Serve - 5002
3. Mail - 5001
4. Flask - 8501
5. Prometheus - 9090
6. Grafana - 3000
7. Model Logging - 8001
8. MLFlow UI - 5000
9. Redis - 6379
10. Airflow - 8080
11. Alert Manager - 9093
12. Node Manager - 9100

Docker

Services:

1. Model Server
2. Topic Model Serving
3. Sentiment Model Serving
4. Prometheus
5. Grafana
6. Mail Server
7. Airflow
8. Redis
9. Postgres
10. Node Exporter
11. MLFlow UI
12. Alert Manager